数组扁平化

```
[1,[2,[3,[4]]]] → [1,2,3,4]

1、内部递归
const _flatten = arr => {
    let resArr = [];
    const fn = arr => {
        for(let num of arr){
            if(Array.isArray(num)){
                fn(num);  //每次内部递归，操作的是同一个 resArr，固不需要拼接
            }else{
                resArr.push(num);
            }
        }
    }
    fn(arr);
    return resArr;
}

2、外部递归
const _flatten = arr => {
    let resArr = [];
    const fn = arr => {
       for(let num of arr){
           if(Array.isArray(num)){
                resArr = resArr.concat(_flatten(num));  //每次外部递归，操作的是独立作用域下的 resArr，固需要拼接
            }else{
                resArr.push(num);
            }
       }
    }
    fn(arr);
    return resArr;
}

3、借用 toString() 去除中括号的特性
const _flatten = arr => {
    const str = arr.toString();  //"1, 2, 3, 4"
    return str.split(',').map(num => Number(num));
```

```
}

4、借用 reduce()
const _flatten = arr => {
    return arr.reduce((did, cur) => {
        return did.concat(Array.isArray(cur) ? _flatten(cur) : cur);
    }, [])
}
```

找出数组中重复的元素

```
function duplicates(arr) {
    const resArr = [];
    for(let num of arr){
        if(arr.indexOf(num) !== arr.lastIndexOf(num) && resArr.indexOf(num) ===
-1){
            resArr.push(num);
        }
    }
    return resArr;
}

[1, 2, 4, 4, 3, 3, 1, 5, 3] → [1, 3, 4]
```

全排列

```
"abc" → [ 'abc', 'acb', 'bac', 'bca', 'cab', 'cba' ]

const _permute = str => {
    if (str.length === 1) return [str];
    const resArr = [];
    for (let s of str) {
        const filterStr = str.split('').filter(item => item !== s).join('');
        _permute(filterStr).forEach(item => {
            resArr.push(s + item);
        })
    }
```

```
    return resArr;
}
console.log(_permute('abc'));

思路:
1、"abc" 的全排 =  ("a" + 全排"bc") + ("b" + 全排"ac") + ("c" + 全排"ac")
```

两数之和

```
var twoSum = function(nums, target) {
    let length = nums.length;
    let map = new Map([[nums[0],0]])

    for(let i = 1; i < length; i++){
        const res = target - nums[i];
        if(map.has(res)){
            return [i,map.get(res)]
        }else{
            map.set(nums[i],i);
        }
    }
};
```

有效的括号

```
var isValid = function(s) {
    if(s.length === 0) return true;
    if([")","]","}"].includes(s[0])) return false;
    let sArr = s.split("");
    const targetArr = [];
    for(let o of sArr){
        if(o === '('){
            targetArr.push(')')
        }else if(o === '['){
            targetArr.push(']')
        }else if(o === '{'){
            targetArr.push('}')
```

```
        }else{  //右括号
            const cur = targetArr.pop();  //当前需要的右括号
            if(o !== cur){
                return false;
            }
        }
    }

    return Boolean(targetArr.length === 0);
};
```

青蛙上楼梯

```
var climbStairs = function(n) {
    let prePre = 0, pre = 0, stemp = 1;
    for(let i = 1; i <= n; i++){
        prePre = pre;
        pre = stemp;
        stemp = pre + prePre;
    }
    return stemp;
```

求最大连续数组之和

```
var maxSubArray = function(nums) {
    let pre = 0;
    let resMax = nums[0];
    nums.forEach(item => {
        pre = Math.max(pre + item, item);  //如果之前的和为负数，则舍弃
        resMax = Math.max(pre, resMax);  //保留出现过的最大值
    })
    return resMax;
};

[-2,1,-3,4,-1,2,1,-5,4] → 6
[5,4,-1,7,8] → 23
```

求买卖股票最佳时机下的利润

```
var maxProfit = function(prices) {
    let minPrice = prices[0];
    let maxIncome = 0;
    prices.forEach(item => {
        minPrice = Math.min(minPrice, item);
        maxIncome = Math.max(maxIncome,item - minPrice);
    })
    return maxIncome;
};

[7,1,5,3,6,4] → 5
[7,6,4,3,1] → 0
```

找出数组中只出现1次的数字

```
//记录次数法：时间复杂度O(n)，空间复杂度O(n)
var singleNumber = function(nums) {
    const obj = {};
    nums.forEach(num => {
        if(obj[num]){
            obj[num] = 0;
        }else{
            obj[num] = 1;
        }
    });
    for(let num in obj){
        if(obj[num]){
            return num;
        }
    }
};

//异或法：时间复杂度O(n)，空间复杂度O(1)
var singleNumber = function(nums) {
    let res = 0
    nums.forEach(num => {
```

```
        res ^= num;
    })
    return res;
};

[4,1,2,1,2] → 4
```

寻找数组 arr 中出现次数大于 arr.length/2 的数字（一定存在）

```
//排序法
var majorityElement = function(nums) {
    nums.sort();
    return nums[Math.floor(nums.length/2)];
};


//持票方法
var majorityElement = function(nums) {
    let curNum = nums[0];
    let count = 1;
    for(let i = 1; i <= nums.length - 1; i++){
        if(curNum === nums[i]){
            count++;
        }else{
            count--;
        }
        if(count === 0){
            curNum = nums[i];
            count = 1;
        }
    }
    return curNum;
};

[2,2,1,1,1,2,2] → 2
```

寻找数组 arr 中出现次数大于 arr.length/3 的数字（一定存在）

```
var majorityElement = function(nums) {
    if(nums.length <= 2){
        return Array.from(new Set(nums));
    }
    nums.sort()
    const length = nums.length;
    let curNum = nums[0];
    let count = 1;
    const arr = [];
    for(let i = 1; i < nums.length; i++){
        if(curNum === nums[i]){
            count++;
        }else{
            count = 0;
        }
        if(count === 0){
            curNum = nums[i];
            count = 1;
        }
        if(count > length/3){
            arr.push(curNum);
            curNum = nums[i];
            count = 1;
        }
    }
    return Array.from(new Set(arr));
};

[3,2,3] → [3]
[1] → [1]
[1,1,1,3,3,2,2,2] → [1,2]
```

将数组的 0 移动到末尾，同时保持其他数顺序不变（仅在原数组上操作）

```
//直接法
var moveZeroes = function(nums) {
    for(let num of nums){
        if(num === 0){
```

```
            nums.splice(nums.indexOf(0),1);
            nums.push(0)
        }
    }
};

//参考冒泡排序法
var moveZeroes = function(nums) {
    for(let i = 0; i < nums.length - 1; i++){
        for(let j = i + 1; j <= nums.length - 1; j++){
            if(nums[i] === 0){
                [nums[i],nums[j]] = [nums[j],nums[i]]
            }
        }
    }
};
```

最大公约数、最小公倍数

```
function gcd(x, y) {
    return y === 0 ? x : gcd(y, x % y);
}

function lcm(a, b) {
    return (a * b) / gcd(a, b);
}
```

返回最大长度无重复数组的长度

```
let arr = [1, 3, 4, 5, 5, 7, 9, 10, 11, 12];

function maxLength(arr) {
    let left = 0, right = 0, max = 0;
    const set = new Set()
    while (right < arr.length) {
        if (!set.has(arr[right])) {
            set.add(arr[right++]);
```

```
            max = Math.max(max, right - left);
        } else {
            set.delete(arr[left++])
        }
    }
    return max;
}

console.log(maxLength(arr))
```

逆时针输出二维数组

```
var spiralOrder = function(matrix) {
    const result = [];
    while (matrix.length > 0) {
        result.push(...matrix.shift());  // 上行
        for (let i = 0; i < matrix.length; i++) {  // 右列
            if (matrix[i].length > 0) {
                result.push(matrix[i].pop());
            }
        }
        if (matrix.length > 0) {   // 下行
            result.push(...matrix.pop().reverse());
        }
        for (let i = matrix.length - 1; i >= 0; i--) {  // 左列
            if (matrix[i].length > 0) {
                result.push(matrix[i].shift());
            }
        }
    }
    return result;
};
```